Networks with structured weights

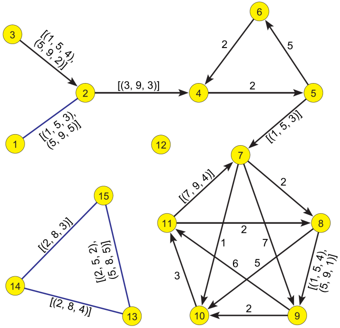
Vladimir Batagelj

IMFM Ljubljana, UP IAM Koper, UL FMF Ljubljana

HSE, Network analysis 2

Zoom, October 20 2025

Outline



Vladimir Batagelj: **vladimir.batagelj@fmf.uni-lj.si**

Last version of slides (October 20, 2025, 05 : 19): StrucNets.pdf

Motivation

In recent years, I used temporal quantities in the analysis of temporal bibliographic [5] and bike-sharing [7] networks. I will put my experiences into a broader framework of networks with structured weights.

Networks

A *network* $\mathbf{N} = (V, L, P, W)$ consists of a *graph* $\mathbf{G} = (V, L)$, where

- V is the set of *nodes*
- $L = E \cup A$, $E \cap A = \emptyset$ is the set of *links* (A is the set of *arcs* – directed links, E is the set of *edges* – undirected links); $n = |V|$, $m = |L|$.
- P are node value functions / *properties*: $p : V \to R_p$; and
- W are link value functions / *weights*: $w : L \to R_w$.

Sometimes, implicit additional information/data about values is provided in the specifications of properties: (a) algebraic structures, and (b) properties of values.

The simplest way to describe a network $\mathbf{N}$ is by providing (V, P) and (L, W) in a form of two tables.

Networks with structured values in R

- The network description in the R package igraph is also based on the two-tables approach.
- A data frame is a rectangular list. Each element (column) of the list has the same length, and each row has a "row name". Each column has its own class, but the class of one column can be different from the class of another column [22, p. 55].
- In R, data frames are supported as data.frame, data.table, and tibble. All of them support structured values.
- To save/load a data frame with structured values, we can use functions saveRDS/readRDS or toJSON/fromJSON (exchange and archiving).
- We are developing a netsJSON format for network description.

Networks with structured values in R


```
> setwd("C:/Users/Public/Sunbelt25")
> library(igraph); library(jsonlite); library(httr)
> source(paste0("https://raw.githubusercontent.com/bavla/",
+   "Nets/refs/heads/master/netsWeight/netsWeight.R"))
> G <- graph_from_literal(Ann-+Chris, Ann-+Bill, Ann+-Dan, Dan+-Chris, Bill++Dan)
> V(G)$x <- c(2,2,8,8); V(G)$y <- c(8,2,8,2)
> G$name <- "Struct1"; G$tit <- "Structured weights - test 1"
> G$by <- "Vladimir Batagelj"; G$date <- date()
> V(G)$cy <- vector("list",4)
> V(G)$cy[[1]] <- c("SI", "US", "FR"); V(G)$cy[[4]] <- c("UK")
> V(G)$cy[[2]] <- c("US", "UK"); V(G)$cy[[3]] <- c("IT", "SI")
> G
IGRAPH 60e8c4f DN-- 4 6 -- Struct1
+ attr: name (g/c), tit (g/c), by (g/c), date (g/c), name (v/c), x
| (v/n), y (v/n), cy (v/x)
+ edges from 60e8c4f (vertex names):
[1] Ann  ->Chris Ann  ->Bill  Chris->Dan   Bill ->Dan   Dan  ->Ann   Dan  ->Bill
> plot(G)
> graph_attr(G)
$name
[1] "Struct1"
$tit
[1] "Structured weights - test 1"
$by
[1] "Vladimir Batagelj"
$date
[1] "Sun Oct 19 18:05:06 2025"
> (nodes <- as_data_frame(G,what="vertices"))
       name x y          cy
Ann     Ann 2 8 SI, US, FR
Chris Chris 2 2     US, UK
Bill   Bill 8 8     IT, SI
Dan     Dan 8 2         UK
> saveRDS(G,file="Struct1.rds")
> write_graph_netsJSON(G,file="Struct1.json")
```

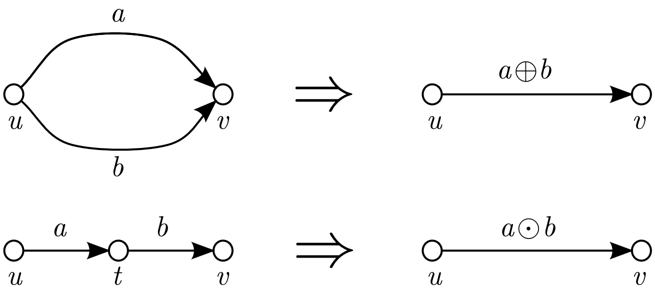
Networks with structured values in R

```
> V(G)$cy[[3]] <- matrix(1:4,nrow=2)
> V(G)$cy[[2]] <- data.frame(b=c(3,7),e=c(5,14),v=c("P","Q"))
> V(G)$cy[[4]] <- list(m=TRUE,k=35)
> (nodet <- as_data_frame(G,what="vertices"))
       name x y                  cy
Ann     Ann 2 8         SI, US, FR
Chris Chris 2 2 3, 7, 5, 14, P, Q
Bill   Bill 8 8         1, 2, 3, 4
Dan     Dan 8 2              1, 35
> nodet$cy[2][[1]]
  b  e v
1 3  5 P
2 7 14 Q
> toString(nodet$cy[2][[1]])
[1] "c(3, 7), c(5, 14), c(\"P\", \"Q\")"
> write_graph_netsJSON(G,file="Testr.json")

> H <- netsJSON_to_graph(fromJSON("Testr.json"),directed=TRUE)
> (nodeh <- as_data_frame(H,what="vertices"))
> nodeh$cy[[1]]
     [,1] [,2] [,3]
[1,] "SI" "US" "FR"
> nodeh$cy[[2]][[1]]
  b  e v
1 3  5 P
2 7 14 Q
> nodeh$cy[[3]][1,,]
     [,1] [,2]
[1,]    1    3
[2,]    2    4
> nodeh$cy[[4]]
     m  k
1 TRUE 35

> library(tidyverse)
> T <- as_tibble(nodeh)
> glimpse(T)
> T
```

Computing with link weights in networks



A *semiring* is a "natural" algebraic structure to formalize computations with link weights in networks. They allow us to extend the weights from links to nodes, walks (paths), and sets of walks.

Semiring

Let $\mathbb{A}$ be a set and a, b, c elements from $\mathbb{A}$. A *semiring* [12, 3, 18, 2, 1] is an algebraic structure $(\mathbb{A}, \oplus, \odot, 0, 1)$ with two binary operations (addition $\oplus$ and multiplication $\odot$) where:

$(\mathbb{A}, \oplus, 0)$ is an *abelian monoid* with the neutral element 0 (zero):
$a \oplus b = b \oplus a$ – commutativity
$(a \oplus b) \oplus c = a \oplus (b \oplus c)$ – associativity
$a \oplus 0 = a$ – existence of zero

$(\mathbb{A}, \odot, 1)$ is a *monoid* with the neutral element 1 (unit):
$(a \odot b) \odot c = a \odot (b \odot c)$ – associativity
$a \odot 1 = 1 \odot a = a$ – existence of a unit

Multiplication $\odot$ *distributes* over addition $\oplus$:
$a \odot (b \oplus c) = a \odot b \oplus a \odot c \qquad (b \oplus c) \odot a = b \odot a \oplus c \odot a$

In formulas, we assume the precedence of multiplication over addition.

Semiring

We can extend the weight w to walks and sets of walks on the network $\mathbf{N}$ by the following rules:

- Let $\sigma_v = (v)$ be a null walk in the node $v \in V$; then $w(\sigma_v) = 1$.
- Let $\sigma = (u_0, u_1, u_2, \ldots, u_{p-1}, u_p)$ be a walk of length $p \geq 1$ on $\mathbf{N}$; then

$$w(\sigma) = \bigodot_{t=1}^{p} w(u_{t-1}, u_t).$$

- For the empty set of walks $\emptyset$, it holds $w(\emptyset) = 0$.
- Let $S = \{\sigma_1, \sigma_2, \ldots\}$ be a set of walks in $\mathbf{N}$; then

$$w(S) = \bigoplus_{\sigma \in S} w(\sigma).$$

Semiring

A semiring $(\mathbb{A}, \oplus, \odot, 0, 1)$ is *complete* iff the addition is well defined for countable sets of elements and the commutativity, associativity, and distributivity hold in the case of countable sets.

These properties are generalized in this case; for example, the distributivity takes the form

$$(\bigoplus_i a_i) \odot (\bigoplus_j b_j) = \bigoplus_i (\bigoplus_j (a_i \odot b_j)) = \bigoplus_{i,j} (a_i \odot b_j)$$

The addition is *idempotent* iff $a \oplus a = a$ for all $a \in \mathbb{A}$. In this case the semiring over a finite set $\mathbb{A}$ is complete.

Semiring

A semiring $(\mathbb{A}, \oplus, \odot, 0, 1)$ is *closed* iff for the additional (unary) *closure* operation $*$ it holds for all $a \in \mathbb{A}$:

$$a^* = 1 \oplus a \odot a^* = 1 \oplus a^* \odot a.$$

Different closures over the same semiring can exist. A complete semiring is always closed for the closure $a^* = \bigoplus_{i \in N} a^i$.

In a closed semiring we can also define a *strict closure* $\overline{a}$ as $\overline{a} = a \odot a^*$.

In a semiring $(\mathbb{A}, \oplus, \odot, 0, 1)$ the *absorption* law holds iff for all $a, b, c \in \mathbb{A}$:

$$a \odot b \oplus a \odot c \odot b = a \odot b.$$

For the absorption law, because of the distributivity, it is sufficient to check the property $1 \oplus c = 1$ for all $c \in \mathbb{A}$.

Semiring

Let $\mathcal{M}_{mn}(\mathbb{A})$ be a set of matrices of order $m \times n$ over the semiring $(\mathbb{A}, \oplus, \odot, 0, 1)$ in which we additionally require

$$\forall a \in \mathbb{A} : a \odot 0 = 0 \odot a = 0,$$

and let $\mathcal{M}(\mathbb{A})$ be a set of all matrices over the $\mathbb{A}$. The operations $\oplus$ and $\odot$ can be extended to the $\mathcal{M}(\mathbb{A})$:

$$\mathbf{A}, \mathbf{B} \in \mathcal{M}_{mn}(\mathbb{A}) : \mathbf{A} \oplus \mathbf{B} = [a_{uv} \oplus b_{uv}] \in \mathcal{M}_{mn}(\mathbb{A})$$

$$\mathbf{A} \in \mathcal{M}_{mk}(\mathbb{A}), \mathbf{B} \in \mathcal{M}_{kn}(\mathbb{A}) : \mathbf{A} \odot \mathbf{B} = [\bigoplus_{t=1}^{k} a_{ut} \odot b_{tv}] \in \mathcal{M}_{mn}(\mathbb{A}).$$

Let $\mathbf{W}$ be a network weight matrix. Matrices over a semiring form a semiring for matrix addition and multiplication.

The entry w_{uv}^k of k-th power $\mathbf{W}^k$ of a weight matrix $\mathbf{W}$ is equal to the value of the set S_{uv}^k of all walks of length k from node u to node v:

$$w(S_{uv}^k) = \mathbf{W}^k[u, v] = w_{uv}^k$$

Fletcher's algorithm

In a complete semiring, also the closure matrix $\mathbf{W}^*$ exists and can be computed using the *Fletcher's algorithm* [17, 13]

$\mathbf{C}_0 \leftarrow \mathbf{W}$
for $k \in 1, \ldots, n$ **do**
  for $i \in 1, \ldots, n$ **do for** $j \in 1, \ldots, n$ **do**
    $c_k[i,j] \leftarrow c_{k-1}[i,j] \oplus c_{k-1}[i,k] \odot c_{k-1}[k,k]^* \odot c_{k-1}[k,j]$
  $c_k[k,k] \leftarrow 1 \oplus c_k[k,k]$
$\mathbf{W}^* \leftarrow \mathbf{C}_n$

In an absorptive semiring $c_{k-1}[k,k]^* = 1$, $c_k[k,k] = 1$, and because of the addition's idempotence, we can compute in place – omit subscripts in the algorithm.

Some semirings used in network data analysis

1. Combinatorial: $(\mathbb{N}, +, \cdot, 0, 1)$ or $(\mathbb{R}_0^+, +, \cdot, 0, 1)$
2. Reachability: $(\{0, 1\}, \vee, \wedge, 0, 1)$
3. Shortest paths [2]: $(\mathbb{R}_0^+, \min, +, \infty, 0)$
4. MaxMin (capacity): $(\mathbb{R}_0^+, \max, \min, 0, \infty)$
5. Pathfinder [21, 23, 24]: $(\overline{\mathbb{R}}_0^+, \min, \boxed{r}, \infty, 0)$ where $a \boxed{r} b = \sqrt[r]{a^r + b^r}$ (Minkowski)
6. Regular languages [25] – regular algebra (union, concatenation, and iteration of sets of strings over an alphabet A).

Histograms

7 Let the set of bins $\mathbf{B} = \{B_1, B_2, \ldots, B_k\}$ be a partition of the set B such that $(\mathbf{B}, \circ)$ is a semigroup. A *histogram* $h : \mathbf{B} \to \mathbb{N}$
$h_i = h(B_i) = |\{X : v(X) \in B_i\}|$

$$h \oplus g = h + g \quad (h \oplus g)(i) = h(i) + g(i)$$

$h \odot g = h * g$ convolution [14, 11]

$$(h * g)(i) = \sum_{p \circ q = i} h(p) \cdot g(q)$$

Temporal quantities

8 A temporal quantity (TQ) a is a function $a : \mathcal{T} \to A \cup \{\maltese\}$ where $\maltese$ denotes the value *undefined*. $(A, +, \cdot, 0, 1)$ is a semiring. The *activity time set* T_a of a consists of instants $t \in T_a$ in which a is defined $T_a = \{t \in \mathcal{T} : a(t) \in A\}$.

We can extend both operations to the set $A_\maltese = A \cup \{\maltese\}$ by requiring that for all $a \in A_\maltese$ it holds $a + \maltese = \maltese + a = a$ and $a \cdot \maltese = \maltese \cdot a = \maltese$.
The structure $(A_\maltese, +, \cdot, \maltese, 1)$ is also a semiring.

Let $A_\maltese(\mathcal{T})$ denote the set of all TQs over $A_\maltese$ in time $\mathcal{T}$. To extend the operations to networks and their matrices we first define the *sum* (parallel links) $a + b$ as [6]

$$(a+b)(t) = a(t) + b(t) \quad \text{and} \quad T_{a+b} = T_a \cup T_b.$$

The *product* (sequential links) $a \cdot b$ is defined as

$$(a \cdot b)(t) = a(t) \cdot b(t) \quad \text{and} \quad T_{a \cdot b} = T_a \cap T_b.$$

Temporal quantities

Let us define TQs $\mathbf{0}$ and $\mathbf{1}$ with requirements $\mathbf{0}(t) = ⌘$ and $\mathbf{1}(t) = 1$ for all $t \in \mathcal{T}$. Again, the structure $(A_⌘(\mathcal{T}), +, \cdot, \mathbf{0}, \mathbf{1})$ is a semiring.

To produce a software support for computation with TQs we limit it to TQs that can be described as a sequence of disjoint time intervals with a constant value

$$a = [(s_i, f_i, v_i)]_{i \in 1..k}$$

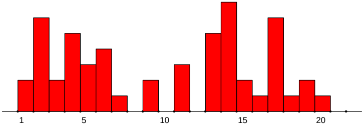
where s_i is the starting time and f_i the finishing time of the i-th time interval $[s_i, f_i)$, $s_i < f_i$ and $f_i \leq s_{i+1}$, and v_i is the value of a on this interval (over combinatorial semiring). Outside the intervals, the value of TQ a is undedined, ⌘.

See also [20].

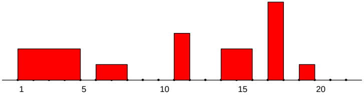
Another approach based on semirings [19, 15].

Sum and product of temporal quantities

$a + b$:

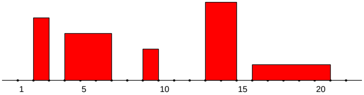


a :



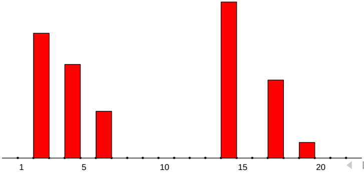
$$a = [(1,5,2),(6,8,1),(11,12,3),(14,16,2)(17,18,5),(19,20,1)$$

b :



$$b = [(2,3,4),(4,7,3),(9,10,2),(13,15,5),(16,21,1)]$$

$a \cdot b$:



Temporal quantities

Let $T_V(v) \subseteq \mathcal{T}$, $T_V \in \mathcal{P}$, be the activity set for a node $v \in \mathcal{V}$ and $T_L(\ell) \subseteq \mathcal{T}$, $T_L \in \mathcal{W}$, the activity set for a link $\ell \in \mathcal{L}$. The following *consistency condition* must be fulfilled for activity sets:

> *If a link $\ell(u,v)$ is active at the time point t then its end-nodes u and v should be active at the time point t* : $T_L(\ell(u,v)) \subseteq T_V(u) \cap T_V(v)$.

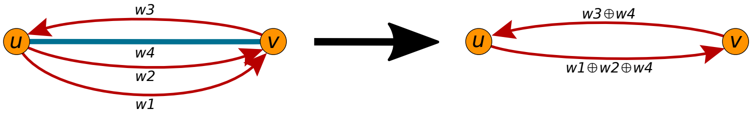
In the following, we will need

1. *Total*: $\text{total}(a) = \sum_i (f_i - s_i) \cdot v_i$
2. *Average*: $\text{average}(a) = \dfrac{\text{total}(a)}{|T_a|}$ where $|T_a| = \sum_i (f_i - s_i)$
3. *Maximum*: $\max(a) = \max_i v_i$

To support the computations with TQs we developed in Python the libraries TQ and Nets, see Github/bavla/TQ.

Simplification of weighted networks

For a subset $S \subseteq L$ of links, we define its *total* weight as $\Sigma(S) = \bigoplus_{\ell \in S} w(\ell)$.



A *simplification* of a weighted network $\mathbf{N} = (V, L, w)$ over an abelian monoid $(\mathbb{A}, \oplus, 0)$ is its transformation into a *simple* directed weighted network $\widehat{\mathbf{N}} = (V, A, \widehat{w})$ such that for

$$L(u, v) = \{\ell \in L : \mathrm{init}(\ell) = u \wedge \mathrm{term}(\ell) = v\}$$

it holds $L(u, v) \neq \emptyset \Rightarrow (u, v) \in A \wedge \widehat{w}(u, v) = \Sigma(L(u, v))$.

For undirected weighted networks, a special undirected simplification can be defined similarly.

Activity of nodes

$$\mathrm{inStar}(v) = \{\ell \in L : \mathrm{term}(\ell) = v\}, \quad \mathrm{outStar}(v) = \{\ell \in L : \mathrm{init}(\ell) = v\}$$

Using the link weight w of the network $\mathbf{N}$ we can compute some interesting node properties such as *in-sum* (weighted indegree):

$$inS(\mathbf{N}, v) = \Sigma(\mathrm{inStar}(v))$$

and *out-sum* (weighted outdegree):

$$outS(\mathbf{N}, v) = \Sigma(\mathrm{outStar}(v))$$

of a node v. They are describing the *activity* of the node v.

Activity of sets

For a simple weighted network $\mathbf{N} = (V, A, w)$ and a subset of nodes $C \subseteq V$ we can define the *boundaries* of C:

$$\mathrm{inBd}(C) = \{\ell \in A : \mathrm{init}(\ell) \notin C \land \mathrm{term}(\ell) \in C\}$$
$$\mathrm{outBd}(C) = \{\ell \in A : \mathrm{init}(\ell) \in C \land \mathrm{term}(\ell) \notin C\}$$
$$\mathrm{Bd}(C) = \mathrm{inBd}(C) \cup \mathrm{outBd}(C)$$

the *interior* of C:

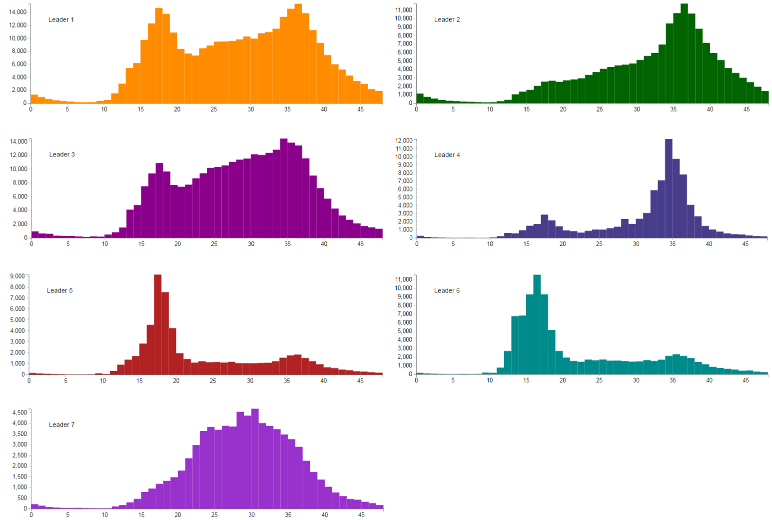
$$\mathrm{Int}(C) = \{\ell \in A : \mathrm{init}(\ell) \in C \land \mathrm{term}(\ell) \in C\}$$

and the corresponding *activities*:

$$\mathrm{inPut}(C) = \Sigma(\mathrm{inBd}(C)) \quad \text{and} \quad \mathrm{outPut}(C) = \Sigma(\mathrm{outBd}(C))$$
$$\mathrm{Act}(C) = \Sigma(\mathrm{Int}(C))$$

Bike-sharing data analysis – Citi bike

Clustering of flows / 7 clusters [7, 4]



NY Citi Bike one-year data from October 2015 to September 2016: 13266296 trips, 678 stations.

Application – temporal bibliographic networks

Let the binary *affiliation* matrix $\mathbf{A} = [a_{ep}]$ describe a two-mode network on the set of events E and the set of of participants P:

$$a_{ep} = \begin{cases} 1 & p \text{ participated at the event } e \\ 0 & \text{otherwise} \end{cases}$$

The function $d : E \to \mathcal{T}$ assigns to each event e the date $d(e)$ when it happened. Assume $\mathcal{T} = [first, last] \subset \mathbb{N}$. Using these data we can construct two temporal affiliation matrices [5]:

- **instantaneous Ai** $= [ai_{ep}]$, where

$$ai_{ep} = \begin{cases} [(d(e), d(e) + 1, 1)] & a_{ep} = 1 \\ [\,] & \text{otherwise} \end{cases}$$

- **cumulative Ac** $= [ac_{ep}]$, where

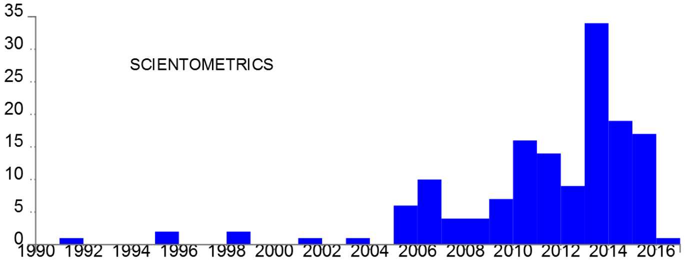
$$ac_{ep} = \begin{cases} [(d(e), last + 1, 1)] & a_{ep} = 1 \\ [\,] & \text{otherwise} \end{cases}$$

Networks are from the collection of bibliographic networks on peer review from WoS till 2017.

The derived network describing citations between journals is obtained as

$$\mathbf{JCJ} = \mathbf{WJi}^T \cdot \mathbf{CiI} \cdot \mathbf{WJc}$$

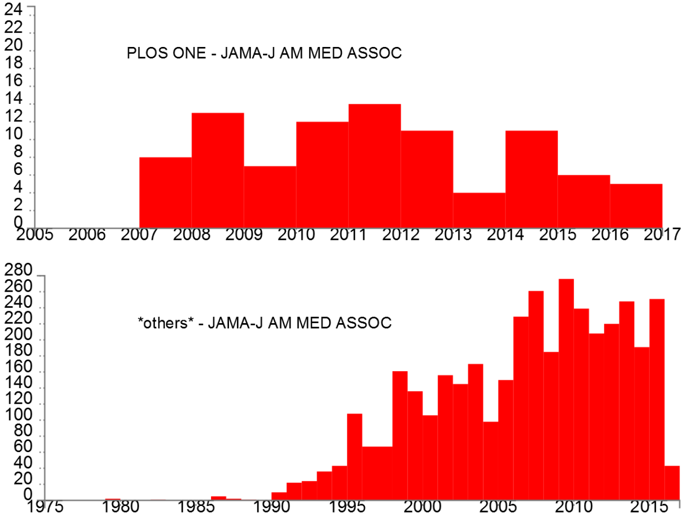
Note that the third network in the product is cumulative.



Self-citations

Temporal bibliographic networks

Temporal citations between journals



Citations between journals

Shortest paths with capacity constraints

About a month ago, I wondered what the shortest path problem is if, for individual links, in addition to the lengths, we also know the load capacities of the bridges on them.

For a vehicle with a given mass w, the problem is equivalent to the shortest path problem on the link cut at the threshold w. For the general solution of the shortest paths between all nodes, I managed to construct an appropriate semiring.

At the seminar, I will present the semiring and the development of software support for solving the shortest path problem with restrictions on the link capacities.

Length and capacity semiring

An arc p in the weighted network (V, L, w) has weight described by a pair $w(p) = (d(p), c(p))$ where $d(p)$ is the length of the arc p and $c(p)$ is the capacity of the arc p. These weights can be extended to the paths and set of paths and described by a sequence of pairs

$$\mathbf{a} = [(ad_1, ac_1), (ad_2, ac_2), \ldots, (ad_k, ac_k)]$$

where ad_i, ac_i are nonnegative real numbers and

$$i < j \Rightarrow (ac_i < ac_j) \wedge (ad_i < ad_j)$$

We will call them the *length and capacity weights* and denote their set with $\mathbf{C}$.

The interpretation of the pair (ad_i, ac_i) is: if the vehicle weight $w \in (ac_{i-1}, ac_i]$ then the shortest path has length ad_i.

Length and capacity semiring

For a given sequence **a** and a weight w, the corresponding pair (ad_i, w) is denoted by $\mathbf{a}(w)$.

We can introduce in **C** two operations defined pointwise for $w \in \mathbb{R}_0^+ \cup \{\infty\}$

addition $(\mathbf{a} \oplus \mathbf{b})(w) = (\min(d(\mathbf{a}(w)), d(\mathbf{b}(w))), w)$ and
multiplication $(\mathbf{a} \odot \mathbf{b})(w) = (d(\mathbf{a}(w)) + d(\mathbf{b}(w)), w)$

Because the pointwise structure $(\mathbb{R}_0^+ \cup \{\infty\}, \min, +, \infty, 0)$ is an absorptive semiring so is also the structure $(\mathbf{C}, \oplus, \odot, \mathbf{0}, \mathbf{1})$, where $\mathbf{0} = [(\infty, \infty)]$ and $\mathbf{1} = [(0, \infty)]$.

$$\mathbf{1} \oplus \mathbf{a} = \mathbf{1}$$

Length and capacity semiring

addition and multiplication of length and capacity weights

```
sumW <- function(A,B){
  na = nrow(A); nb <- nrow(B)
  ia <- 1; ib <- 1; C <- NULL; do <- -1
  repeat {
    if(ia<=na) {da <- A[ia,1]; ta <- A[ia,2]} else {da <- ta <- Inf}
    if(ib<=nb) {db <- B[ib,1]; tb <- B[ib,2]} else {db <- tb <- Inf}
    d <- min(da,db); t <- min(ta,tb)
    if(d==do) C[nrow(C),2] <- t else C <- rbind(C,c(d,t))
    do <- d
    if(t>=ta) ia <- ia+1
    if(t>=tb) ib <- ib+1
    if((ia>na)&(ib>nb)) break
  }
  return(C)
}

mulW <- function(A,B){
  na = nrow(A); nb <- nrow(B)
  ia <- 1; ib <- 1; C <- NULL; do <- -1
  while((ia<=na)&(ib<=nb)) {
    ta <- A[ia,2]; tb <- B[ib,2]
    d <- A[ia,1]+B[ib,1]; t <- min(ta,tb)
    if(d==do) C[nrow(C),2] <- t else C <- rbind(C,c(d,t))
    do <- d
    if(t>=ta) ia <- ia+1
    if(t>=tb) ib <- ib+1
  }
  return(C)
}

> A <- rbind( c(7,15), c(15,20), c(18,30), c(28,Inf) )
> B <- rbind( c(11,20), c(14,30), c(24,40), c(31,Inf) )
> Z <- rbind( c(Inf,Inf) );  E <- rbind( c(0,Inf) )
> sumW(A,Z)               > mulW(A,E)               > sumW(A,E)               > mulW(A,Z)
     [,1] [,2]                 [,1] [,2]                 [,1] [,2]                 [,1] [,2]
[1,]    7   15            [1,]    7   15            [1,]    0  Inf            [1,]  Inf  Inf
[2,]   15   20            [2,]   15   20
[3,]   18   30            [3,]   18   30
[4,]   28  Inf            [4,]   28  Inf
```

Length and capacity semiring

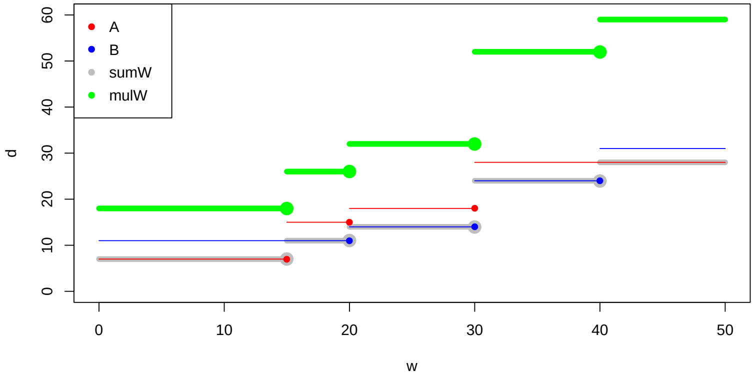
addition and multiplication of length and capacity weights

```
> S <- sumW(A,B); R <- mulW(A,B)
> A
     [,1] [,2]
[1,]    7   15
[2,]   15   20
[3,]   18   30
[4,]   28  Inf

> B
     [,1] [,2]
[1,]   11   20
[2,]   14   30
[3,]   24   40
[4,]   31  Inf

> S
     [,1] [,2]
[1,]    7   15
[2,]   11   20
[3,]   14   30
[4,]   24   40
[5,]   28  Inf

> R
     [,1] [,2]
[1,]   18   15
[2,]   26   20
[3,]   32   30
[4,]   52   40
[5,]   59  Inf
```

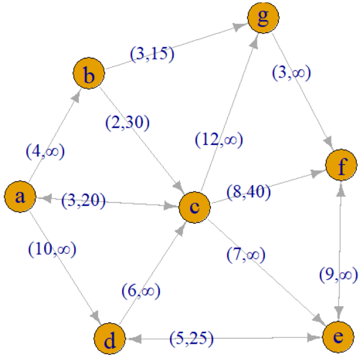


Length and capacity semiring

example network

```
> setwd("C:/Users/vlado/work/R/semi")
> source("capacity.R")
> library(igraph); library(data.table)
> N <- readRDS("semiT2.rds")
> n <- gorder(N); m <- gsize(N)
> nodes <- as_data_frame(N,what="vertices")
> links <- as_data_frame(N,what="edges")
```

```
> nodes
  name   x   y
a    a  70 210
b    b 184 391
c    c 358 194
d    d 218   0
e    e 594   3
f    f 599 256
g    g 471 473
> links
   from to weight      cw dir
1     a  b      1  4, Inf   a
2     a  c      1   3, 20   e
3     a  d      1 10, Inf   a
4     b  c      1   2, 30   a
5     d  c      1  6, Inf   a
6     b  g      1   3, 15   a
7     d  e      1   5, 25   e
8     c  g      1 12, Inf   a
9     c  e      1  7, Inf   a
10    c  f      1   8, 40   a
11    e  f      1  9, Inf   e
12    g  f      1  3, Inf   a
13    c  a      1   3, 20   e
14    e  d      1   5, 25   e
15    f  e      1  9, Inf   e
```




```
> sapply(graph_attr_names(N),function(x) graph_attr(N,x))
```

Length and capacity semiring

Closure

```
> setwd("C:/Users/vlado/work/R/semi")
> source("capacity.R")
> library(igraph); library(data.table)
> CW <- function(p) rbind(C$cw[p][[1]])
> N <- readRDS("semiT2.rds")
> n <- gorder(N); nodes <- as_data_frame(N,what="vertices")
> Z <- rbind(c(Inf,Inf)); E <- rbind(c(0,Inf))
> # initialize closure matrix
> L <- CJ(nodes$name,nodes$name); ZZ <- vector("list",n*n)
> for(i in 1:n**2) ZZ[[i]] <- Z
> C <- data.frame(from=L$V1,to=L$V2); C$cw <- ZZ
> for(p in 1:m){ uv <- as.vector(ends(N,p,names=FALSE))
+   C$cw[(uv[1]-1)*n + uv[2]] <- E(N)$cw[p]
+ }
> # Fletcher's algorithm
> for(t in 1:n){
+   for(u in 1:n) for(v in 1:n) { uv <- (u-1)*n + v
+     ut <- (u-1)*n + t; tv <- (t-1)*n + v
+     C$cw[uv][[1]] <- sumW(CW(uv),mulW(CW(ut),CW(tv)))
+   }
+   tt <- (t-1)*n + t; C$cw[tt][[1]] <- sumW(E,CW(tt))
+ }
```

Length and capacity semiring

Closure

```
> C
   from to                                              cw
1     a  a                                          0, Inf
2     a  b                                          4, Inf
3     a  c                         3,  6,  16,  20, 30, Inf
4     a  d                                         10, Inf
5     a  e                     10, 13,  23,  20,  30, Inf
6     a  f 10,  11,  14,  24,  31, 15,  20,  30,  40, Inf
7     a  g            7,  15,  18, 28,  15,  20,  30, Inf
8     b  a                               5, Inf,  20, Inf
9     b  b                                          0, Inf
10    b  c                               2, Inf,  30, Inf
11    b  d                              14, Inf,  25, Inf
12    b  e                               9, Inf,  30, Inf
13    b  f                     6, 10, Inf,  15,  30, Inf
14    b  g                     3, 14, Inf,  15,  30, Inf
15    c  a                               3, Inf,  20, Inf
16    c  b                               7, Inf,  20, Inf
17    c  c                                          0, Inf
18    c  d                         12, Inf,  25, Inf
19    c  e                                          7, Inf
20    c  f                               8,  15,  40, Inf
21    c  g                              10,  12,  15, Inf
22    d  a                               9, Inf,  20, Inf
.....
34    e  f                                          9, Inf
35    e  g                    21, 23, Inf,  15,  25, Inf
36    f  a                              23, Inf,  20, Inf
37    f  b                              27, Inf,  20, Inf
38    f  c                              20, Inf,  25, Inf
39    f  d                              14, Inf,  25, Inf
40    f  e                                          9, Inf
41    f  f                                          0, Inf
42    f  g                    30, 32, Inf,  15,  25, Inf
43    g  a                         26, Inf,  20, Inf
44    g  b                         30, Inf,  20, Inf
45    g  c                         23, Inf,  25, Inf
46    g  d                         17, Inf,  25, Inf
47    g  e                                         12, Inf
48    g  f                                          3, Inf
49    g  g                                          0, Inf
```

```
> C$cw[6]
[[1]]
     [,1] [,2]
[1,]   10   15
[2,]   11   20
[3,]   14   30
[4,]   24   40
[5,]   31  Inf
> C$cw[46]
[[1]]
     [,1] [,2]
[1,]   17   25
[2,]  Inf  Inf
```

Length and capacity semiring

Extended closure

To be able to reconstruct also the minimal paths, we extend the pairs (d, c) to triples (d, c, t) - the pair (d, c) is realized by passing through the node t. Node 0 indicates a direct link.

Extended operations

```
sumT <- function(A,B){
  na = nrow(A); nb <- nrow(B)
  ia <- 1; ib <- 1; C <- NULL; do <- -1
  repeat {
    if(ia<=na) {da <- A[ia,1]; ta <- A[ia,2]; ka <- A[ia,3]} else {da <- ta <- Inf}
    if(ib<=nb) {db <- B[ib,1]; tb <- B[ib,2]; kb <- B[ib,3]} else {db <- tb <- Inf}
    if(da<db) {d <- da; k <- ka} else {d <- db; k <- kb}
    t <- min(ta,tb)
    if(d==do) {C[nrow(C),2] <- t; C[nrow(C),3] <- k} else C <- rbind(C,c(d,t,k))
    do <- d
    if(t>=ta) ia <- ia+1
    if(t>=tb) ib <- ib+1
    if((ia>na)&(ib>nb)) break
  }
  return(C)
}

mulT <- function(A,B,k){
  na = nrow(A); nb <- nrow(B)
  ia <- 1; ib <- 1; C <- NULL; do <- -1
  while((ia<=na)&(ib<=nb)) {
    ta <- A[ia,2]; tb <- B[ib,2]
    d <- A[ia,1]+B[ib,1]; t <- min(ta,tb)
    if(d==do) C[nrow(C),2] <- t else C <- rbind(C,c(d,t,k))
    do <- d
    if(t>=ta) ia <- ia+1
    if(t>=tb) ib <- ib+1
  }
  return(C)
}
```

Length and capacity semiring

Extended closure

```
closureT <- function(N){
  n <- gorder(N); m <- gsize(N)
  nodes <- as_data_frame(N,what="vertices")
  Z <- rbind(c(Inf,Inf)); E <- rbind(c(0,Inf))
# initialize closure matrix
  L <- CJ(nodes$name,nodes$name)
  ZZ <- vector("list",n**2)
  for(i in 1:n**2) ZZ[[i]] <- cbind(Z,0)
  C <- data.frame(from=L$V1,to=L$V2); C$cw <- ZZ
  for(p in 1:m){ uv <- as.vector(ends(N,p,names=FALSE))
    C$cw[(uv[1]-1)*n + uv[2]] <- list(cbind(rbind(E(N)$cw[p][[1]]),0))
  }
# Fletcher's algorithm
  for(t in 1:n){
    for(u in 1:n) for(v in 1:n) { uv <- (u-1)*n + v
      ut <- (u-1)*n + t; tv <- (t-1)*n + v
      C$cw[uv][[1]] <- sumT(CW(C,uv),mulT(CW(C,ut),CW(C,tv),t))
    }
    tt <- (t-1)*n + t; C$cw[tt][[1]] <- sumT(cbind(E,0),CW(C,tt))
  }
  return(C)
}

> N <- readRDS("semiT2.rds")
> n <- gorder(N)
> CX <- closureT(N)
> nodes <- as_data_frame(N,what="vertices")
> CNx <- graph_from_data_frame(CX,directed=TRUE,vertices=nodes)
> CNx$name <- "semiT2 extended closure"
> CNx$tit <- "Extended closure of the Test network 2 for the capacity semiring"
> CNx$by <- "Vladimir Batagelj"
> CNx$cdate <- date()
> saveRDS(CNx,file="xclosureT2.rds")
> write_graph_netsJSON(CNx,file="xclosureT2.json")
```

Length and capacity semiring

Extended closure

```
> CX
   from to                                                 cw
1     a  a                                          0, Inf, 0
2     a  b                                          4, Inf, 0
3     a  c                     3, 6, 16, 20, 30, Inf, 0, 2, 4
4     a  d                                         10, Inf, 0
5     a  e                   10, 13, 23, 20, 30, Inf, 3, 3, 4
6     a  f 10, 11, 14, 24, 31, 15, 20, 30, 40, Inf, 7, 3, 3, 4, 7
7     a  g             7, 15, 18, 28, 15, 20, 30, Inf, 2, 3, 3, 4
8     b  a                                5, Inf, 20, Inf, 3, 7
.....
20    c  f                             8, 15, 40, Inf, 0, 7
.....
24    d  c                                          6, Inf, 0
.....
27    d  f                            14, 21, 40, Inf, 3, 7
.....
42    f  g                  30, 32, Inf, 15, 25, Inf, 5, 5, 7
.....
48    g  f                                          3, Inf, 0
49    g  g                                          0, Inf, 0
> rbind(V(N)$name)
     [,1] [,2] [,3] [,4] [,5] [,6] [,7]
[1,] "a"  "b"  "c"  "d"  "e"  "f"  "g"
> CW(CX,6)
     [,1] [,2] [,3]
[1,]   10   15    7
[2,]   11   20    3
[3,]   14   30    3
[4,]   24   40    4
[5,]   31  Inf    7
> # w=40 1/a-4/d-6/f;  a-d > 4 link; d-f > 27
> CW(CX,27)
     [,1] [,2] [,3]
[1,]   14   40    3
[2,]   21  Inf    7
> # w=40 4/d-3/c-6/f; d-c > 24 link; c-f > 20
> CW(CX,20)
     [,1] [,2] [,3]
[1,]    8   40    0
[2,]   15  Inf    7
> # 1/a-4/d-3/c-6/f -> 10+6+8 = 24
```

Length and capacity semiring

Path reconstruction

```
CW <- function(C,p) rbind(C$cw[p][[1]])
nodex <- function(C,u,v,c){
  uv <- (u-1)*n + v; T <- CW(C,uv)
  if(T[1,1] == Inf) return(0)
  for(i in 1:nrow(T)) if(T[i,2]>=c) break
  if(T[i,1] == Inf) return(0)
  return(T[i,3])
}
.path <- function(C,u,v,c){
  k <- nodex(C,u,v,c)
  if(k == 0) return(NULL)
  return(c(.path(C,u,k,c),k,.path(C,k,v,c)))
}
path <- function(C,u,v,c){
  if(nodex(C,u,v,c) == 0) return(NULL)
  return(c(u,.path(C,u,v,c),v))
}
paths <- function(C,u,v){
  uv <- (u-1)*n + v; T <- CW(C,uv)
  P <- data.frame(d=T[,1],c=T[,2]); P$P <- vector("list",nrow(T))
  for(i in 1:nrow(T)) P$P[i] <- list(path(C,u,v,T[i,2]))
  return(P)
}

> (P <- paths(CX,1,6))
   d   c             P
1 10  15    1, 2, 7, 6
2 11  20       1, 3, 6
3 14  30    1, 2, 3, 6
4 24  40    1, 4, 3, 6
5 31 Inf 1, 4, 3, 7, 6
> Q <- P
> for(i in 1:nrow(P)) Q$P[i] <- list(nodes$name[P$P[i][[1]]])
> Q
   d   c             P
1 10  15    a, b, g, f
2 11  20       a, c, f
3 14  30    a, b, c, f
4 24  40    a, d, c, f
5 31 Inf a, d, c, g, f
```

Conclusions

Work in progress.

Most of the approaches to temporal network analysis are cross-sectional – based on time slices.

Another approach is based on event sequences. In applications, an event has additional properties besides the pair of nodes and starting time, such as duration, type, location, etc. [8].

The longitudinal approach is an alternative to be considered.

Collections of interesting, understandable, and well-documented datasets (bikes, world trade, bibliographic data, political events (KEDS, WEIS), etc.). Sources of problems and ideas.

New analytical procedures: temporal clustering of nodes, temporal 1-neighbors, temporal cores, etc [9].

Acknowledgments

The computational work reported in this presentation was performed using R. The code and data are available at GitHub/bavla/semirings.

This work was partially supported by the Slovenian Research Agency (research program P1-0294 and research project J5-4596) and was prepared within the framework of COST action CA21163 (HiTEc).

References I

Abdali SK, Saunders BD (1985) Transitive closure and related semiring properties via eliminants. Theor Com-put Sci 40:257—274

Baras JS, Theodorakopoulos G (2010) Path problems in networks. Morgan & Claypool, Berkeley

Batagelj V (1994) Semirings for social networks analysis. J Math Soc 19(1):53-–68

Batagelj V, Kejžar N (2019) clamix: Clustering modal multi-valued symbolic data. rForge

Batagelj V, Maltseva D(2020) Temporal bibliographic networks. Journal of Informetrics, 14 (1), 101006.

Batagelj V, Praprotnik S (2016) An algebraic approach to temporal network analysis based on temporal quantities. Social Network Analysis and Mining 6(1): 1–22.

References II

- Batagelj, V (2017) Symbolic network analysis of bike-sharing data – Citi Bike. 6th Symbolic Data Analysis Workshop, Ljubljana, 12–14. June 2017. slides
- Batagelj, V, Ferligoj, A, Oštro Center (2024) Event sequence analysis - Network analysis of sequences of events. 1343+1344. sredin seminar, Ljubljana, March 6, 2024. slides
- Batagelj, V.: Clustering and Blockmodeling Temporal Networks - Two Indirect Approaches. In: Paula Brito, José G. Dias, Berthold Lausen, Angela Montanari, Rebecca Nugent (eds.) Classification and Data Science in the Digital Age. Proceedings of IFCS 2022, Porto. Springer 2023, paper, book.
- Brubaker B (2025) New Method Is the Fastest Way To Find the Best Routes. Quanta magazine, August 6, 2025. WWW
- Brunotte H (2013) Discrete Convolution Rings. Romanian Journal of Mathematics and Computer Science, Volume 3, Issue 2, p.155–159

References III

- Carré B (1979) Graphs and networks. Clarendon, Oxford
- Cerinšek M, Batagelj V (2017) Semirings and Matrix Analysis of Networks. In: Alhajj R., Rokne J. (eds) Encyclopedia of Social Network Analysis and Mining. Springer, New York, NY.
- Dongol B, Hayes IJ, Struth G (2016) Convolution as a Unifying Concept: Applications in Separation Logic, Interval Calculi and Concurrency ACM Transactions on Computational Logic, Article No.: 15
- Falzon L, Quintane E, Dunn J, Robins G (2018) Embedding time in positions: Temporal measures of centrality for social network analysis. Social Networks 54, 168–178
- Duan R, Mao J, Mao X, Shu X, Yin L (2025) Breaking the Sorting Barrier for Directed Single-Source Shortest Paths. April 23, 2025. arXiv

References IV

- Fletcher JG (1980) A more general algorithm for computing closed semiring costs between vertices of a directed graph. Commun ACM 23(6):350–351
- Gondran M, Minoux M (2008) Graphs, dioids and semirings: new models and algorithms. Springer, New York
- Kontoleon N, Falzon L, Pattison P (2013) Algebraic structures for dynamic networks. Journal of Mathematical Psychology 57, 310–319
- Praprotnik S, Batagelj V (2016) Semirings for temporal network analysis. arXiv:1603.08261 [cs.SI]
- Schvaneveldt RW, Dearholt DW, Durso FT (1988) Graph theoretic foundations of Pathfinder networks. Comput Math Appl 15(4):337–345
- Stack Overflow (2018) R Notes for Professionals book. PDF

References V

- Quirin A, Cordón O, Santamaria J, Vargas-Quesada B, Moya-Anegón F (2008) A new variant of the Pathfinder algorithm to generate large visual science maps in cubic time. Inf Process Manag 44(4): 1611–1623
- Vavpetič A, Batagelj V, Podpečan V (2009) An implementation of the Pathfinder algorithm for sparse networks and its application on text networks. Proceedings of the 12th international multiconference Information Society, Volume A, 236–239. PDF
- Wikipedia: Regular language.